

A Feedback-Driven, Multi-Model LLM Prompt Optimizer: Treating Prompt Engineering as an Automated Debugging Loop

CS460-560 Automated Debugging — Project #6: Using LLMs for Prompt Engineering

Authors: Duygu Görgün (29525), Beyzagül Demir (38564) **Institution:** Sabancı University **Course / Instructor:** CS460-560 Automated Debugging — Cemal Yılmaz **Term:** Spring 2025-2026

2. Abstract

Large Language Models (LLMs) are extremely sensitive to the wording of their prompts, and a prompt that works on one model frequently fails on another — especially on smaller or differently-trained models. Manually patching a prompt until it behaves correctly on a *fleet* of models is exactly the observe–diagnose–patch cycle that this course studies for ordinary programs. In this project we build a system that treats **prompt engineering as an automated debugging loop**. Given (1) an initial, intentionally weak prompt, (2) a list of target LLMs, and (3) a benchmark of `<input, expected_output>` test cases, the system iteratively rewrites the prompt until every target model passes every test case (or a configurable threshold / iteration budget is reached). The loop is implemented as a LangGraph state machine with four stages — **run on training set**, **run on held-out validation set**, **analyze failures**, **optimize prompt** — driven by a strong "optimizer" LLM that consumes a structured failure report and produces a revised prompt without hard-coding test answers. We evaluate the system on four self-contained benchmark suites spanning structured JSON extraction, tool/function-call routing, locale-aware numerical reasoning, and **vision** (screenshot) bug reporting. On a fully **free** model configuration (all models accessed through the OpenRouter gateway at \$0 cost), the optimizer lifted every benchmark from a weak baseline of **0–35 %** combined pass rate to **100 %** within a single optimization step (two iterations), with no generalization gap on the held-out validation cases. We report concrete before/after model outputs, per-iteration convergence curves, the failure modes the optimizer fixed, and an honest discussion of the threats to the study.

3. Introduction

Prompt engineering has become a core software-construction activity: the "source code" of an LLM-powered feature is, increasingly, a natural-language prompt. Yet prompts are brittle. The **same** instruction can produce a perfectly-formatted JSON object on a 120B model and an unparseable paragraph of prose on a 9B model. As teams deploy a single prompt across a **portfolio** of models (different sizes, vendors, and quantization levels, chosen for cost or latency reasons), the engineer faces a debugging problem: **which models fail, on which inputs, and how must the prompt change so that all of them pass — without overfitting to the examples?**

This is structurally identical to the debugging cycle taught in CS460-560: **reproduce** the failure, **observe** the discrepancy between expected and actual behavior, **localize** the cause, and **apply a fix** that does not regress the cases that already worked. The novelty is that both the program under repair (the prompt) and the repair tool (the optimizer) are LLMs.

Problem statement

Input 1 — an initial prompt (user-provided; specific and moderately complex).

Input 2 — a list of target LLM models.

Input 3 — a benchmark: **N** test cases, each `{input, expected_output}`.

Output — an updated prompt that passes all benchmark test cases on **all** target models, expressed in the **most general** form possible (it must not merely memorize the test answers; it must generalize to unseen inputs).

Contributions

- A **file-driven, model-agnostic optimizer** in which a new benchmark is added by dropping files into a folder — no code changes are required.
- A **LangGraph feedback loop** with an explicit **held-out validation set**, **best-prompt tracking** (anti-regression), and a **generalization-gap** monitor that guards against the classic over-fitting failure of prompt optimizers.
- A **programmatically verifiable** evaluation design (JSON key/value matching plus a locale-aware numeric comparator) that avoids the cost and noise of an LLM-as-judge.
- **Four diverse benchmarks**, including a genuinely hard **locale-aware reasoning** task and a **multimodal (vision)** task, all driven to 100 % on a **free** model configuration, with concrete before/after evidence and a documented version-by-version improvement process.

4. Related Work and Background

Our design sits in the now-active area of **automatic prompt optimization (APO)**. The references below were verified against their primary sources; arXiv identifiers are included.

- **Automatic Prompt Engineer (APE)**. Zhou et al., *"Large Language Models Are Human-Level Prompt Engineers,"* ICLR 2023 (arXiv:2211.01910). APE treats the instruction as a "program," has an LLM **propose** a pool of candidate instructions, and **selects** the best by a score function. Our optimizer node is a directed (failure-guided) analogue of APE's propose-and-select: instead of sampling many candidates blindly, we feed the optimizer a structured failure report so each proposal is a targeted repair.
- **Optimization by PROMpting (OPRO)**. Yang, Wang, Lu, Liu, Le, Zhou, Chen, *"Large Language Models as Optimizers,"* ICLR 2024 (arXiv:2309.03409). OPRO frames the LLM as a gradient-free optimizer that is shown previous solutions and their scores and asked for better ones. Our loop is an instance of this idea specialized to the **multi-model, pass/fail** setting: the "objective value" is the per-model pass rate and the "trajectory" is the prompt history.
- **Automatic Prompt Optimization with "Gradient Descent" and Beam Search (ProTeGi/APO)**. Pryzant, Iter, Li, Lee, Zhu, Zeng, *"EMNLP 2023"* (arXiv:2305.03495). APO forms natural-language "gradients" that criticize the current prompt and edits the prompt in the opposite semantic direction. Our **failure report** is exactly such a natural-language gradient — it tells the optimizer **which** model failed, on **which** case, and the **expected-vs-actual delta**.
- **DSPy**. Khattab, Singhvi, Maheshwari, et al., *"DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,"* 2023 (arXiv:2310.03714). DSPy compiles LM pipelines and bootstraps few-shot demonstrations to maximize a metric. We share DSPy's philosophy of **optimizing toward a metric**,

but keep the artifact a single human-readable system prompt (rather than a compiled pipeline) so the output is directly auditable by a developer.

- **Promptbreeder.** Fernando, Banarse, Michalewski, Osindero, Rocktäschel, *"Promptbreeder:*

Self-Referential Self-Improvement via Prompt Evolution,"* 2023 (arXiv:2309.16797). Promptbreeder **evolves** a population of prompts with LLM-generated mutation operators. It is a useful contrast: our method is a **gradient-style** single-line-of-descent optimizer (cheaper, more interpretable) rather than a population-based evolutionary search.

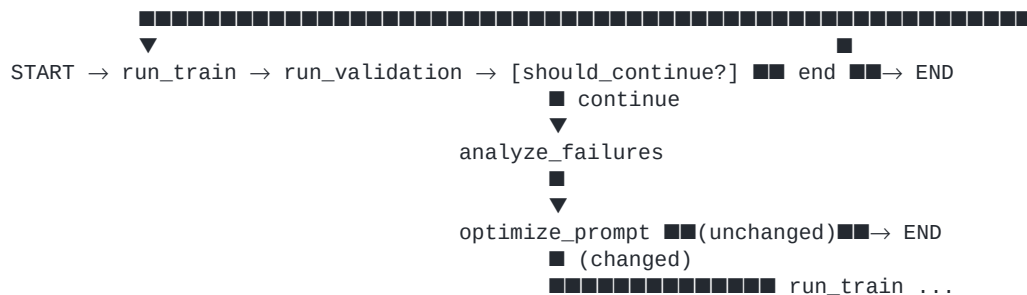
Background tooling. The loop is orchestrated with **LangGraph** (a typed state-machine library from the LangChain project) and uses the **OpenAI-compatible** client to reach every model through the **OpenRouter** gateway, which exposes many open-weight and commercial models behind a single API key. These are engineering substrates rather than research contributions; we cite their documentation in the References.

Positioning. Relative to the above, the distinguishing features of our system are: (a) the optimization target is **simultaneous** correctness across a **heterogeneous set of target models** (the "worst model" governs convergence), (b) a **held-out validation split** that the optimizer never sees, used both to **report** a generalization gap and to **select** the best prompt, and (c) first-class **multimodal** (vision) and **locale-aware** benchmarks.

5. Approach

5.1 System overview

The system is a closed feedback loop. A **weak prompt** enters; on each iteration it is run against every target model on the training cases, scored, and — if it has not yet reached the threshold — the failures are summarized and handed to a strong **optimizer LLM** that returns a rewritten prompt. The loop repeats until success or the iteration budget is exhausted.



5.2 The four nodes (LangGraph state machine)

Implemented in `optimizer_graph.py` over a typed `OptimizerState` (a `TypedDict`).

- `run_train` — sends the current prompt + each training input to every target model

(`temperature = 0`) and scores the responses with the evaluator.

- `run_validation` — runs the **same** prompt on a held-out validation set (never shown to

the optimizer), records the per-iteration history, computes a case-count-weighted **combined** pass rate, and updates the **best prompt** seen so far (anti-regression: even if a later iteration is worse, the best prompt is preserved and ultimately saved).

- `analyze_failures` — builds a structured **failure report**: for every failing

(model, test case) pair it lists whether the output was valid JSON, which required keys were missing, and the actual-vs-expected delta (plus per-key value mismatches in value-match mode).

- `optimize_prompt` — calls the optimizer LLM with a fixed **meta-prompt** plus the current

prompt and the failure report, and returns a revised prompt. If the optimizer returns an identical prompt, the loop stops (stagnation guard).

Two conditional edges control flow: `should_continue` (stop on threshold reached or budget exhausted) and `check_prompt_changed` (stop if the optimizer made no change).

5.3 Evaluator (programmatic, no LLM-as-judge)

`llm_utils.py` parses each model response into JSON — tolerating markdown code fences, stray prose, and `<think>...</think>` reasoning blocks — and scores it in one of two modes:

- `keys_only` — the score is the fraction of **required keys present** in a valid JSON object.

Used for `bug_reporter`, `tool_caller`, and `screenshot_reporter`, where the schema (not the exact value) is the contract.

- `value_match` — the score additionally requires the **values** to match within a similarity

threshold. Used for `locale_query`. A dedicated `_parse_number` routine understands both US (1,000.50) and Turkish (1.000,50) locale conventions, and numeric comparison tolerates a small epsilon. This is what lets us grade "is the computed answer actually correct?" without an expensive judge model.

5.4 The meta-prompt and the generalization guard

The optimizer is steered by a **meta-prompt** (`llm_utils.META_PROMPT`) that instructs it to: diagnose **systematic** patterns rather than one-off mistakes; add explicit output-format rules (exact keys, no markdown, no prose); reason about number formats when a numeric answer is wrong by a consistent factor; **build on** the rules that already pass; and — critically — **not hard-code any expected output value**. Over-fitting is further contained by three mechanisms inherited from the project plan: (1) a **20 % held-out validation split** that the optimizer never sees, (2) the explicit anti-hard-coding instruction, and (3) a **generalization gap** readout (`train - val`) printed every iteration.

5.5 File-driven project abstraction

`project_loader.py` turns a folder into a runnable benchmark. A project directory contains `index.md` (train/validation split), `credentials.md` (provider, target models, optimizer model, thresholds, match mode, call delay), `prompt.md` (the weak starting prompt), and `tests/<id>/{input.*, expected_output.json}`. Inputs may be **text** (.txt/.md), **images** (.png/.jpg, base64-encoded into a multimodal message), or **documents** (.pdf, text is extracted with `pdfplumber`). Adding a new benchmark therefore requires **zero** code changes.

5.6 Development process (version-by-version)

The codebase records a clear evolution from a throwaway probe to a productized loop:

- **v0 — Motivating probes.** `benchmark_test.py` runs one weak prompt across several models to

show the divergence (the same prompt scores very differently per model); `benchmark_improved.py` contrasts the weak prompt with a hand-written improved one — establishing that a better prompt closes the gap and motivating automation.

- **v1 — Self-contained loop.** `optimizer_loop.py` implements the runner → evaluator → failure

report → optimizer cycle with hard-coded models, prompt, and benchmark. This is the reference implementation of the core algorithm.

- **v2 — Project abstraction.** `project_loader.py` + `llm_utils.py` externalize prompts,

models, and test cases into files, decoupling the engine from any specific task.

- **v3 — LangGraph state machine.** `optimizer_graph.py` re-expresses the loop as a typed graph

with conditional edges, adds the **held-out validation** node, **best-prompt tracking**, and the **generalization-gap** monitor.

- **v4 — Value-level grading.** The `value_match` mode and locale-aware `_parse_number` were

added so the harder `locale_query` task could be graded on `*answer correctness*`, not just schema.

- **v5 — Multimodal + documents.** Image (vision) and PDF inputs were added to the loader,

enabling the `screenshot_reporter` benchmark.

- **v6 — Free-model migration (this report).** The originally-configured target models had

become unavailable or rate-limited on the free tier; we migrated the entire study to a set of currently-available :free OpenRouter models (Section 6.1) and re-ran every benchmark from scratch.

6. Experiments

6.1 Experimental setup

All models are reached through the OpenRouter OpenAI-compatible gateway. The configuration below is **entirely free** (every model carries the `:free` suffix; a full run costs \$0 in API credits). Calls use `temperature = 0`, a 3-second inter-call delay, and exponential back-off retry on HTTP 429 (free models are intermittently rate-limited upstream).

Role	Text projects	Vision project (<code>screenshot_reporter</code>)
Target model 1	<code>nvidia/nemotron-3-nano-30b-a3b:free</code>	<code>nvidia/nemotron-nano-12b-v2-vl:free</code>
Target model 2	<code>openai/gpt-oss-20b:free</code>	<code>nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free</code>
Optimizer	<code>openai/gpt-oss-120b:free</code>	<code>openai/gpt-oss-120b:free</code>

The two target models per project are deliberately **heterogeneous** (different vendors and behaviors) so that a single weak prompt fails them in **different** ways — the realistic multi-model debugging scenario.

Reproducibility note. The originally-submitted configuration used

`nvidia/nemotron-nano-9b-v2:free` and `z-ai/glm-4.5-air:free`. By the time of this report the

latter was no longer offered for free (HTTP 404, "use the paid slug") and several popular

Meta/Qwen/Google :free models were returning HTTP 429. We therefore migrated to the models

above, which were verified available, and re-ran the full study. The optimized prompts are

generic, so they transferred without task-specific changes.

6.2 The four benchmarks (summary, I/O, and mechanism)

(a) bug_reporter — free-text bug report → 5-field JSON (text, keys_only) *Mechanism.* The model reads a free-form bug report and must emit a JSON object with exactly {severity, affected_component, affected_platform, condition, reproducibility}. *Example input:* "The login button on the checkout page crashes the app on iOS 17.2 when the user has more than 5 items in their cart. Happens 100 % of the time." *Expected output:* {"severity": "high", "affected_component": "checkout/login_button", "affected_platform": "iOS 17.2", "condition": "cart_items > 5", "reproducibility": "100%"}.

Split: 4 train + 2 validation.

(b) tool_caller — request + tool list → tool-call JSON (text, keys_only) *Mechanism.* Given a user request and a list of available tools, the model must select the right tool and emit {tool_name, arguments}. *Example input:* "Send a meeting reminder to john@example.com with subject 'Team Sync' and body 'Meeting at 3pm today'" (with three available tools). *Expected output:* {"tool_name": "send_email", "arguments": {"to": "john@example.com", "subject": "Team Sync", "body": "Meeting at 3pm today"}}.

Split: 3 train + 1 validation.

(c) locale_query — mixed-locale table QA → answer JSON (text, value_match) *Mechanism.* The table uses the **Turkish** convention where . is the *thousands* separator, so 1.500 means *one thousand five hundred*. The model must detect the locale, normalize the numbers, compute the answer, and emit {answer} as a plain digit string. *Example input:* a parts table with weights 1.500, 2.250, 3.100 and the question "What is the total weight of all parts?" *Expected output:* {"answer": "6850"} (= 1500 + 2250 + 3100). Split: 4 train + 2 validation. This is the hardest task: it requires *correct reasoning*, not just schema compliance.

(d) screenshot_reporter — app screenshot → 5-field JSON (**vision**, keys_only) *Mechanism.* A PNG screenshot of a crashing/erroring app is passed as a multimodal message; the model must emit {severity, affected_component, affected_platform, error_type, reproducibility}. Split: 3 train + 1 validation.

6.3 Convergence results

The table reports the **combined** (case-count-weighted train+val) pass rate at the first iteration (the weak prompt) versus the best iteration, on the free-model configuration. "Step" counts optimizer rewrites (iteration 2 is the result of a single rewrite of the weak prompt).

Project	Weak prompt (iter 1) train / val / combined	Best (iter 2) train / val / combined	Optimizer steps
bug_reporter	45 % / 15 % / 35 %	100 % / 100 % / 100 %	1
tool_caller	25 % / 25 % / 25 %	100 % / 100 % / 100 %	1
locale_query	0 % / 0 % / 0 %	100 % / 100 % / 100 %	1
screenshot_reporter	10 % / 10 % / 10 %	100 % / 100 % / 100 %	1

Every benchmark reaches **100 %** — comfortably above the project's ≥ 90 % target — with a **0-point generalization gap** (the held-out validation cases pass at the same rate as the training cases), indicating the optimizer produced a *general* rule rather than memorizing the training answers. Convergence in a single step reflects how informative the structured failure report is: the failures are *systematic* (wrong schema, wrong locale), so one well-targeted rewrite fixes them across all cases.

6.4 Qualitative before/after evidence

For one representative case per project we recorded the **raw** output of each target model under the weak prompt and under the optimized prompt (full records in projects/<name>/results/sample_outputs.json).

These expose the *failure modes* the optimizer diagnosed and repaired.

bug_reporter / tc_001 (expected the 5 canonical keys):

- *Before* — nemotron-3-nano-30b invents its own schema:

```
{"component":"Login      button...","platform":"iOS","os_version":"17.2","trigger_condition":...,  
"behavior":"App crashes",...} (score 0.4). gpt-oss-20b wraps the JSON in a ``` `json` fence and adds  
non-schema keys (bug_id, title, module, ...) (score 0.4).
```

- *After* — both emit exactly

```
{"severity":"high","affected_component":"checkout/login_button","affected_platform":"iOS  
17.2", "condition":"cart_items > 5","reproducibility":"100%"} (score 1.0).
```

tool_caller / tc_002:

- *Before* — nemotron uses the key "tool" instead of "tool_name" (score 0.5); gpt-oss-20b
returns an **empty** completion (score 0.0).

- *After* — both emit

```
{"tool_name":"send_email","arguments":{"to":"john@example.com","subject":"Team      Sync",  
"body":"Meeting at 3pm today"}} (score 1.0).
```

locale_query / tc_001 (the hard, value-graded task):

- *Before* — both models misread the Turkish thousands separator and treat 1.500 as the decimal
1.5, answering 6.85 / "6.85" (score 0.0).
- *After* — both correctly normalize the locale and answer {"answer":"6850"} (score 1.0). The
optimizer's learned rule generalizes to the validation table (3.200 + 1.800 + 2.500 → "7500").

screenshot_reporter / tc_001 (vision):

- *Before* — nemotron-nano-12b-v2-v1 returns a paragraph of **prose** describing the screenshot
(not valid JSON, score 0.0); the omni model invents a schema (error_message, error_code, cart_items,
total, ...) (score 0.2).

- *After* — both emit exactly

```
{"severity":"high","affected_component":"checkout","affected_platform":"iOS      17.2",  
"error_type":"crash","reproducibility":"100%"} (score 1.0).
```

6.5 What the optimizer changed (anatomy of a repair)

Comparing each weak prompt with its optimized version (projects/<name>/results/optimized_prompt.md) shows a consistent repair strategy: (i) state the **exact** required keys and their order; (ii) **forbid** prose, markdown fences, and extra keys; (iii) give one **illustrative example using made-up values** (never a test value); and, for locale_query, (iv) add an explicit **number-format rule** ("values use . as a thousands separator → multiply by 1000 and emit a plain digit string"). Notably, the optimized prompts contain **no** test-case answer — consistent with the anti-hard-coding instruction and confirmed by the zero generalization gap.

6.6 Operational observations

- **The weak prompt fails differently per model**, validating the multi-model premise: e.g., on

`tool_caller` the same weak prompt yielded a near-miss key error on one model and an empty response on the other.

- **Free-tier reliability** is the main practical obstacle: popular `:free` models return HTTP

429 under load. The client's exponential back-off plus a 3-second inter-call delay made full runs complete reliably on the `nvidia-Nemotron` and `openai-gpt-oss` families.

- **Reasoning models** (e.g., the `omni` reasoning target) occasionally emit empty content if the

token budget is consumed by hidden reasoning; the `strip_thinking` filter and a generous `max_tokens` mitigate this.

7. Threats to Study (Threats to Validity)

- **Small benchmarks.** Each suite has 4–6 cases. 100 % is therefore strong evidence of the

mechanism working but not a precise accuracy estimate; confidence intervals would require more cases. The held-out split (≥ 1 –2 cases) limits, but does not eliminate, optimism.

- **Convergence in one step is partly task-driven.** Because the failures are highly systematic

(schema/locale), a single rewrite suffices. Tasks with *heterogeneous, case-specific* failures would likely need more iterations and could expose over-fitting that our small validation set cannot detect.

- **Learned rules can over-specialize.** The `locale_query` fix ("multiply by 1000") is correct

for the benchmark's `x.xxx-thousands` format and generalizes within it, but it is *not* a fully general locale parser; a table mixing genuine decimals and thousands could break it. This is an honest limitation of optimizing against a finite benchmark.

- **Model non-determinism and availability.** Even at `temperature = 0`, free-tier providers can

vary outputs and availability over time; the exact numbers are reproducible only against the same model snapshots. The original target models had already drifted out of the free tier, forcing a mid-project migration.

- `keys_only` **grading is lenient.** For three of four tasks we grade *schema*, not *values*; a

model could emit a correctly-keyed but semantically wrong field and still score 1.0. The `value_match` mode used for `locale_query` shows the stricter regime is feasible but costlier to author.

- **Single optimizer model.** All repairs were produced by one optimizer (`gpt-oss-120b:free`).

A weaker or different optimizer might converge more slowly or differently; we did not run an optimizer ablation in this configuration.

- **No human-judged tasks.** We deliberately restricted to programmatically verifiable outputs

to avoid LLM-as-judge cost and noise; open-ended generation quality is out of scope.

8. Concluding Remarks and Future Work

We showed that prompt engineering can be cast as an automated debugging loop and that a modest LangGraph state machine — runner, validator, failure analyzer, optimizer — can repair a weak prompt so that a *fleet* of heterogeneous, **free** LLMs all pass a benchmark. Across four diverse tasks (structured extraction, tool routing, locale-aware reasoning, and a multimodal vision task) the system moved every benchmark from a 0–35 % baseline to 100 % with no generalization gap, using a held-out validation split and an explicit anti-over-fitting design.

Future work.

- **Larger, harder benchmarks** with 30–50 cases and adversarial/edge inputs, to measure accuracy with confidence intervals and to stress the generalization guard.
- **Optimizer and quantization ablations** — compare optimizers and explicitly include different *quantization levels* of one base model as targets (the original plan's idea), since the worst-case quantized model is the true test of generality.
- **Stronger generalization guards** — a regression guard that rejects any rewrite which lowers *any* model's pass rate, plus stagnation detection over *N* iterations.
- **Value-level grading everywhere** — extend `value_match` (and a general locale parser) to all tasks so schema-correct-but-wrong outputs are penalized.
- **Response caching and parallel calls** to cut wall-clock time and free-tier rate-limit exposure during large sweeps.
- **Prompt-diff reporting** — surface a structured diff of what changed each iteration to make the "debugging" narrative explicit to the developer.

9. References

- Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, J. Ba. *Large Language Models Are Human-Level Prompt Engineers (APE)*. ICLR 2023. arXiv:2211.01910.
- C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, X. Chen. *Large Language Models as Optimizers (OPRO)*. ICLR 2024. arXiv:2309.03409.
- R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, M. Zeng. *Automatic Prompt Optimization with "Gradient Descent" and Beam Search (APO/ProTeGi)*. EMNLP 2023. arXiv:2305.03495.
- O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, et al. *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*. 2023. arXiv:2310.03714.
- C. Fernando, D. Banarse, H. Michalewski, S. Osindero, T. Rocktäschel. *Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution*. 2023. arXiv:2309.16797.
- LangChain, Inc. *LangGraph: Building stateful, multi-actor applications with LLMs*.

Documentation: <https://langchain-ai.github.io/langgraph/>.

- OpenRouter. *A unified API gateway for LLMs.* <https://openrouter.ai/docs>.
- OpenAI. *OpenAI Python library (OpenAI-compatible client).*

<https://github.com/openai/openai-python>.

Artifacts accompanying this report: full source (cli.py, optimizer_graph.py, optimizer_loop.py, llm_utils.py, project_loader.py), the four benchmark suites under projects/, per-run logs (results/run_log.txt), per-iteration logs (results/optimization_log.json), the optimized prompts (results/optimized_prompt.md), the captured before/after outputs (results/sample_outputs.json), and setup/run instructions in README.txt.